

La API de páginas de Mateu

Cómo queda la definición de una página tras adoptar el modelo de los page templates de Redwood / Visual Builder (pack `oj-sp`)

Propuesta — para leer con calma

Fuente: `design/vb-page-template-apis.md` (análisis de la API real de 38 componentes `oj-sp 2604.1.0`, extraída del CDN oficial de Oracle)

30 de julio de 2026

1. La idea en una página

El análisis de las APIs reales de los templates `oj-sp` arrojó una conclusión: no son 38 APIs distintas, sino **una gramática de 6 partes que cada template instancia**. Mateu ya tiene el 80% de esa gramática, pero dispersa. La propuesta no es añadir features: es **completar y unificar**, de forma 100% aditiva (cero rupturas), para que cada página Mateu se defina siempre igual:

Una página Mateu = ARQUETIPO	(la familia: qué template es)
+ HEADER canónico	(título, badges, KPIs, timestamp, peer-nav, switcher, acciones)
+ SLOTS	(métodos overridables, vocabulario cerrado con nombre y rol)
+ DISPLAY OPTIONS	(affordances del arquetipo, interruptores tri-estado)
+ ACCIONES	(actionIds; hooks before* cancelables)
+ ESTADO controlado	(campos del componentState)
+ width / type / density	(ya en el wire)

Cada pieza nueva sigue un idiom que ya existe en la casa:

Pieza nueva	Idiom existente que imita
record en <code>uidl.data</code> (<code>Toggle</code> , <code>RecordSwitcher...</code>)	<code>PeerNav</code> , <code>DateRange</code> , <code>Sort</code>
interfaz <code>*Supplier</code> (<code>RecordSwitcherSupplier</code>)	<code>PeerNavigationSupplier</code> , <code>BannerSupplier</code>
capability opcional (<code>Draftable</code>)	<code>Navigable</code> , <code>Editable</code> , <code>Creatable</code>
método <code>protected</code> overridable con default	<code>emptyDetail()</code> , <code>editInDrawer()</code> , <code>stepApplies()</code>
anotación de clase (<code>@Overline</code>)	<code>@Title</code> , <code>@Compact</code> , <code>@PageWidth</code>

Regla de oro: los `actionIds` siguen siendo el vocabulario de eventos, el `componentState` sigue siendo el estado, y los arquetipos siguen siendo composición de tipos wire existentes. Solo hay 4 campos wire nuevos en toda la propuesta.

2. Vocabulario compartido (uidl.data — tipos nuevos)

Seis tipos pequeños que usan todos los arquetipos. El más importante es `Toggle`: el tri-estado de affordance que oj-sp usa en todas partes y que hoy Mateu no puede expresar («visible pero inerte», útil para permisos).

```
public enum Toggle { on, off, disabled } // affordance tri-estado
public enum Density { standard, compact } // @Compact = azúcar de compact
public enum SwitcherType { context, object }
public enum PanelMode { reflow, overlay } // panel acoplado: empuja vs solapa
public enum Dock { end, bottom }

public record RecordSwitcher(
    List<Option> options, // los registros/contextos elegibles
    String value, // el seleccionado
    SwitcherType type, // context (fija contexto) | object (cambia de registro)
    boolean searchable) {} // con búsqueda si hay muchos

public record DockedPanel(
    Component content,
    Dock dock, // end (lateral) | bottom (inferior)
    PanelMode mode, // reflow (empuja) | overlay (solapa)
    boolean open) {}
```

3. El header canónico

Ya existe casi entero (Fase o del plan RDS). Queda así — lo marcado **NUEVO** es lo único que se añade:

Elemento	API	
Título / subtítulo	@Title / SubtitleSupplier	HOY
Overline (texto sobre el título)	@Overline("Reserva")	NUEVO
Placeholder del título en modo create	@TitlePlaceholder("Nueva reserva")	NUEVO
Avatar / icono	AvatarSupplier / PageDto.icon	HOY
Badges de estado	@BadgeInHeader / BadgeSupplier	HOY
Contextual info (pares clave/valor)	@KPI	HOY
Timestamp («última actualización»)	@Timestamp	HOY
Peer-nav (objeto ← anterior siguiente →)	PeerNavigationSupplier	HOY
Switcher de registro/contexto	RecordSwitcherSupplier	NUEVO
Acciones primaria / secundarias	toolbar + buttonStyle (el arquetipo posee los roles)	HOY
Banners	@Banner / BannerSupplier	HOY
Región aria-live (a11y)	UICommand.announce(text)	NUEVO

3.1 RecordSwitcherSupplier

En oj-sp el switcher de registro es un elemento *de cabecera* disponible en overview, dashboard, colección y búsqueda — no algo interno de un template. Se generaliza igual:

```
public interface RecordSwitcherSupplier {
    RecordSwitcher switcher(HttpRequest rq);
    Object switchTo(String value, HttpRequest rq); // this / URI / State
}
```

`GeneralOverview` pasa a implementarla (su campo `record + switcherOptions()` actuales se convierten en la implementación por defecto) y cualquier página detail/dashboard puede declararla. Wire: `PageDto.switcher`, render en el `mateu-content-header` compartido.

4. `display()` : las affordances del arquetipo, unificadas

La decisión clave. Cada template oj-sp trae sus affordances incorporadas (New, Delete, Export, Save...) y el desarrollador solo las conmuta con `displayOptions`. En Mateu ese patrón ya existe pero disperso: gates `canX()`, anotaciones `@Not*`, overrides boolean sueltos. Se consolida en **un record tipado POR arquetipo + un único método overridable** (no una bolsa genérica de flags — autocompletado y discoverability):

```
// en Crud
public record CrudDisplay(
    Toggle newButton, Toggle delete, Toggle export, Toggle columns,
    Toggle selector, Toggle importButton, Toggle history,
    Toggle saveAndNext, // «guardar y siguiente» (drawer)
    Density density) {
    public static CrudDisplay defaults() { ... } // derivado de canX()/@Not* → cero rupturas
}

public CrudDisplay display() { return CrudDisplay.defaults(); }
```

```
// en Wizard
public record WizardDisplay(
    Toggle saveDraft, Toggle saveAndClose, Toggle skip,
    ChecklistDisplay checklist, // current | all
    Density density) {
    public static WizardDisplay defaults() { ... }
}

protected WizardDisplay display() { return WizardDisplay.defaults(); }
```

- `Toggle.disabled` = visible pero inerte → mapea al `disabled` que `ButtonDto` ya transporta; casa con `@DisabledUnless` (permisos).
- Los mecanismos actuales **no se rompen**: alimentan los defaults del record. `@NotDeletable` ⇒ `defaults().delete == off`.
- Sin tipo wire nuevo: el record se consume server-side al componer la página.

5. Slots: el vocabulario cerrado, formalizado

El modelo Mateu «slot = método overridable del arquetipo» ya es isomorfo al de oj-sp. Se documenta con los nombres canónicos y se añaden los 4 que faltan:

Arquetipo	Slot canónico (oj-sp)	API Mateu	
GeneralOverview	main	overview(Row, rq)	HOY
	info (panel contextual)	info(Row, rq) → compone ContentLayout(main, aside)	NUEVO
CollectionDetail	detailTemplate	detail(Row, rq)	HOY
	noSelection	emptyDetail()	HOY
SmartSearchPage	dashboard (contenido pre-búsqueda)	preSearchContent(rq)	NUEVO
Dashboard	kpi	campos MetricCard → Scoreboard	HOY
Wizard	completionStep	el último paso (resultado)	HOY
Foldout	overview	primer campo sin @Panel	HOY
	summary (contenido de la tira COLAPSADA)	FoldoutPanel.summary(Component)	NUEVO
ItemOverview	overview / main	key-info + @Panel tabs	HOY
DataManagement	innerEnd/outerEnd/innerBottom/outerBottom	endPanel(rq) / bottomPanel(rq) → DockedPanel	NUEVO
drawers	header	Drawer.header	HOY
todos	search, announcement	smart bar / announce()	HOY
			NUEVO

Firmas de los nuevos:

```
// GeneralOverview – la anatomía real del template es main + info
protected Component info(Row row, HttpRequest rq) { return null; }
// si != null: ContentLayout(main = overview, aside = info); display().promoteInfo()

// SmartSearchPage – qué mostrar mientras aún no se ha buscado
protected Component preSearchContent(HttpRequest rq) { return null; }

// Foldout – resumen visible en la tira plegada + orientación
public FoldoutPanel summary(Component c); // fluent
protected Orientation orientation() { return Orientation.horizontal; }

// DataManagement – los 4 paneles acoplados del template real
protected DockedPanel endPanel(HttpRequest rq) { return null; }
protected DockedPanel bottomPanel(HttpRequest rq) { return null; }
```

6. Wizard transaccional (paridad guided-process)

Tres piezas, todas opcionales. `Draftable` es una capability al estilo `Navigable` / `Editable`; los otros dos son métodos junto al `stepApplies()` existente:

```
// uidl.interfaces – capability opcional del wizard
public interface Draftable {
    Object saveDraft(HttpRequest rq);           // botón "Guardar borrador"
    default String resumeStep(HttpRequest rq) { // reanudar donde se dejó
        return null;                           // null = primer paso aplicable
    }
}

// en Wizard
protected boolean stepSkippable(String stepFieldName) { return false; }

protected void beforeStepNavigate(String from, String to, HttpRequest rq)
    throws ValidationException {}           // hook CANCELABLE (validación por paso)
```

La API real de oj-sp confirma este trío por partida triple: `displayOptions.save/saveAndClose + resumeStepId` en `guided-process`, `spSkip{skippedStepId}`, y el `spFinishLater` del `step-by-step-page`. El «guardar y seguir luego» es un patrón de primera clase en Redwood.

7. Retoques menores por arquetipo

Arquetipo	Añadido	Origen oj-sp
Welcome	<code>protected HeroTone heroTone()</code> — enum DS-neutral {ocean, pine, lilac, teal, rose, pebble, slate, plum, sienna, auto}; redwood-oj lo mapea a la paleta RDS, el resto a sus tokens	<code>backgroundColor</code> (9 tonos + auto)
HeroSearch	<code>protected List<Suggestion> suggestions(rq)</code> — chips de sugerencia bajo el buscador	<code>suggestions[]</code> + <code>spSuggestionAction</code>
Crud (drawer)	<code>actionId save-and-next</code> (guardar y abrir el siguiente del listado, combina con el <code>peerNav</code> del drawer) + banner de error estándar al fallar el save en drawer (sin API: comportamiento del framework)	<code>spPrimaryActionAndNext</code> , <code>displayErrorMessageBanner</code>
CalendarPage (futuro)	<code>CalendarDisplay(monthView, weekView, dayView, listView, firstDayOfWeek...)</code> + slot <code>eventDetail(event, rq)</code>	<code>selectedViewValue</code> + <code>displayOptions.*View</code> + template slots
@Section (futuro)	affordances por sección: <code>edit / add / viewMore</code>	componente <code>section</code>

8. Cómo queda para el desarrollador

8.1 Página de detalle con switcher, peer-nav y panel info

```
@UI("/reservas/:id")
@Overline("Reserva")
public class ReservaOverview extends GeneralOverview<Reserva>
    implements PeerNavigationSupplier {           // el switcher ya lo trae el arquetipo

    @Autowired ReservaService service;

    @Override protected Reserva load(String id, HttpRequest rq) {
        return service.byId(id);
    }

    @Override protected Component overview(Reserva r, HttpRequest rq) {    // slot main
        return new EntityHeader(r.titulo(), r.badges(), r.facts(), ...);
    }

    @Override protected Component info(Reserva r, HttpRequest rq) {        // slot info ★
        return new Card("Facturación", propertyList(r.folio()));
    }

    @Override public PeerNav peers(HttpRequest rq) {                        // ← | →
        return service.peersOf(rq.routeValue("id"));
    }
}
```

8.2 Wizard con borradores, skip y validación por paso

```
@UI("/checkin/:id/wizard")
@WizardProgress(WizardProgressStyle.RAIL)
public class CheckInWizard extends Wizard implements Draftable {    // ★

    IdentidadStep identidad;
    PreferenciasStep preferencias;    // saltable
    PagoStep pago;
    ResultadoStep resultado;

    @Override protected WizardDisplay display() {                    // ★
        return WizardDisplay.defaults()
            .with(d -> d.saveDraft(Toggle.on).skip(Toggle.on));
    }

    @Override public Object saveDraft(HttpRequest rq) { ... }        // ★
    @Override public String resumeStep(HttpRequest rq) {            // ★
        return draft().lastStep();
    }

    @Override protected boolean stepSkippable(String step) {        // ★
        return "preferencias".equals(step);
    }

    @Override protected void beforeStepNavigate(String from, String to,
        HttpRequest rq) throws ValidationException {                // ★
        if ("pago".equals(to) && !identidad.completa())
            throw new ValidationException("Completa la identidad antes de pagar");
    }
}
```

```
@WizardCompletionAction
public void confirmar(HttpServletRequest rq) { ... }
}
```

8.3 Crud afinando sus affordances

```
@UI("/facturas")
public class FacturasCrud extends AutoCrud<Factura> {

    @Override public CrudStore<Factura> store() { return store; }

    @Override public CrudDisplay display() {                // ★
        return CrudDisplay.defaults().with(d -> d
            .delete(Toggle.disabled)    // visible pero inerte (permisos lo activan)
            .export(Toggle.on)
            .saveAndNext(Toggle.on)      // edición en serie desde el drawer
            .density(Density.compact));
    }

    @Override public boolean editInDrawer() { return true; }
}
```


9. La tabla de API por arquetipo (el entregable de documentación)

Lo que convierte «15 arquetipos sueltos» en «un sistema»: cada arquetipo se documenta con la **misma tabla de 6 filas** (como hace el Component Exchange de Oracle). Ejemplo:

GeneralOverview<Row> — página de registro (pageType: detail)	
Header	@Title / @Overline, @KPI, @BadgeInHeader, @Timestamp, switcher (incorporado, overridable vía RecordSwitcherSupplier), PeerNavigationSupplier
Slots	overview(Row, rq) [main, obligatorio] · info(Row, rq) [aside, opcional] · emptyOverview() [pre-selección]
Display	OverviewDisplay(promoteInfo, switcherSearch, density)
Acciones	switchRecord (incorporada) + las @Action del modelo
Estado	record (registro seleccionado)
Página	width: fixed (default) · type: detail · density vía display()

Wizard — proceso guiado (pageType: process)	
Header	@Title (processTitle), progreso vía @WizardProgress {BAR, STEPS, RAIL}
Slots	un campo WizardStep por paso · el último paso = completionStep (resultado)
Display	WizardDisplay(saveDraft, saveAndClose, skip, checklist, density)
Acciones	next / previous / skip (incorporadas), @WizardCompletionAction; hooks: stepApplies, stepSkippable, beforeStepNavigate; capability Draftable
Estado	paso actual + estado de cada paso (cross-step, persiste)
Página	width: fixed · type: process

10. Impacto, compatibilidad y paridad

10.1 Wire: 4 campos nuevos

- PageDto.overline (String)
- PageDto.titlePlaceholder (String)
- PageDto.switcher (RecordSwitcherDto)
- FoldoutPanelInfoDto.summary (Component slotted)

Los DockedPanel probablemente reutilicen DrawerDto con un campo mode. Todo lo demás (display options, slots, hooks, capabilities) se consume server-side al componer y viaja con los tipos wire existentes.

10.2 Compatibilidad: cero rupturas

- Todos los métodos nuevos tienen default null/off que preserva el comportamiento actual.
- canX() / @Not* / @Compact siguen funcionando: alimentan los defaults de los records display().
- Ningún constructor ni interfaz existente cambia de firma.

10.3 Paridad de puertos

Java	.NET	Python
<code>Toggle</code> , <code>CrudDisplay</code> record	<code>Toggle</code> , <code>CrudDisplay</code> record + <code>virtual Display()</code>	<code>Toggle</code> enum + <code>dataclass</code> + <code>display()</code>
<code>RecordSwitcherSupplier</code>	<code>IRecordSwitcherSupplier</code>	<code>mixin RecordSwitcherSupplier</code>
<code>Draftable</code>	<code>IDraftable</code>	<code>mixin Draftable</code>
<code>@Overline</code> / <code>@TitlePlaceholder</code>	<code>[Overline]</code> / <code>[TitlePlaceholder]</code>	<code>@overline(...)</code> / <code>@title_placeholder(...)</code>

11. Orden de implementación sugerido

Cada entrega = release + demo + SyncTest + doc + paridad .NET/Python (el patrón de siempre):

1. **Vocabulario** + `display()` en Crud y Wizard (la pieza de gramática; el resto de arquetipos después, uno a uno).
2. **RecordSwitcherSupplier** + refactor de GeneralOverview sobre él (+ los 2 Strings del header: `@Overline`, `@TitlePlaceholder`).
3. **Slot info** de GeneralOverview (compone el `ContentLayout` ya existente).
4. **Wizard transaccional**: `Draftable` + `stepSkippable` + `beforeStepNavigate`.
5. **save-and-next** del drawer + banner de error estándar.
6. **Foldout** `summary / orientation`; **SmartSearchPage** `preSearchContent`; **Welcome** `heroTone`; **HeroSearch** `suggestions`.
7. **DockedPanel** en DataManagement (la pieza con más incógnitas de frontend — al final).
8. **La tabla de API por arquetipo** en la doc de usuario (§9) — puede empezar YA con lo existente e ir incorporando lo nuevo.

Y lo que NO se hace: props `DataProvider` client-side (Mateu es server-driven), bolsas de `translations` por componente (`Translator` es transversal), feedback survey / undo-redo / dashboards editables por el usuario final (piezas grandes de Fusion Apps, solo bajo demanda), y los canvases de autoría (`canvas-page`, `visual-space-page`, `data-authoring`).